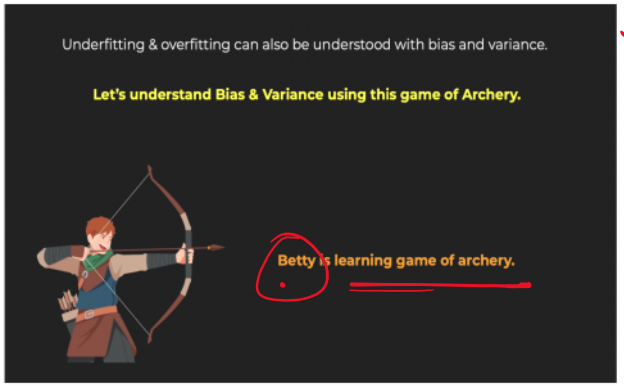
Wednesday, May 20, 2026 3:35 PM

Regularization Recap ⇒

⑦ ⇒ Regularization

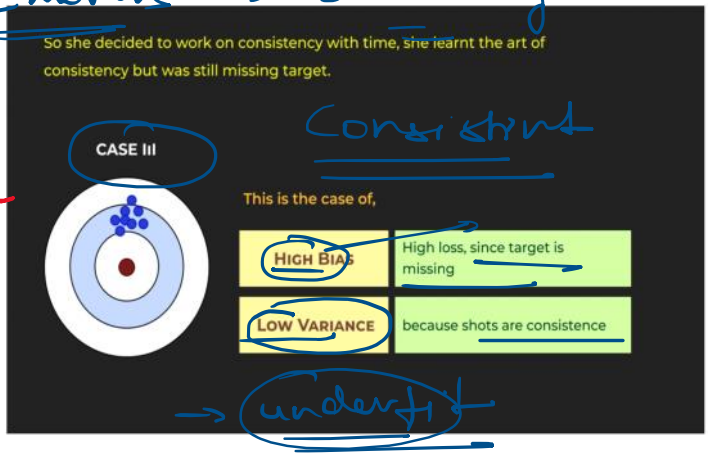
- ① Train ↑ (90%), Test ↓ (30%) → Overfitting
- ② Train ↓, Test ↓ → Underfitting
- ③ Train ↑, Test ↑ → Perfect model
- ④ Train ↓, Test ↑

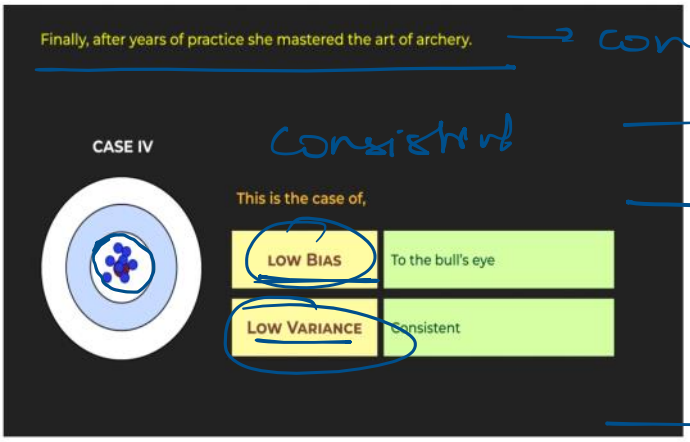
Bias Variance Trade off ⇒











Consistency + hitting near target

Consistent

Required

low Bias + low variance

Bias & variance ⇒ -ve things

Required → low Bias + low variance + ideal model

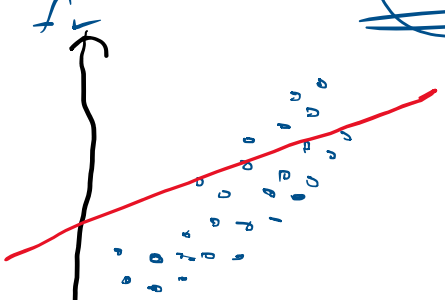
Bias → Training DATA ERROR $(y - \hat{y})$

↳ are u able to fit DATA well or not

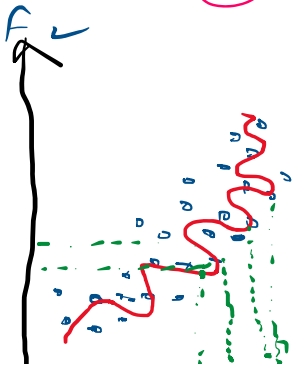
↳ Train

2 cases ⇒

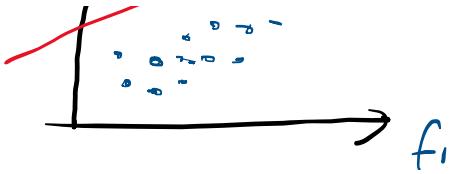
Bias



make model



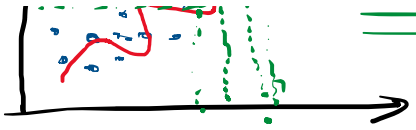
NO consistency



* High Bias

ERROR

VARIANCE ↑



Low Bias

Bias → $y - \hat{y}$

* Train & Test

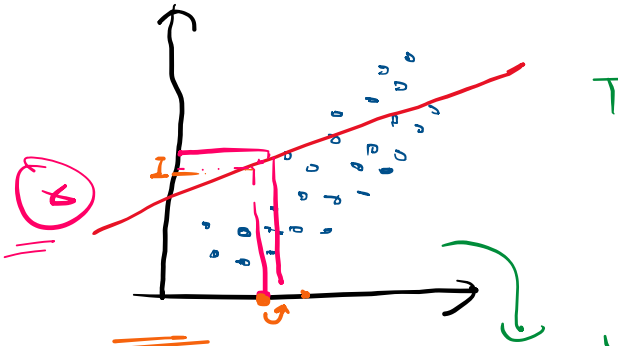
↑ → 90%

↓ 30%. 20%

VARIANCE → spreadness / variance of model

→ How much the model varies

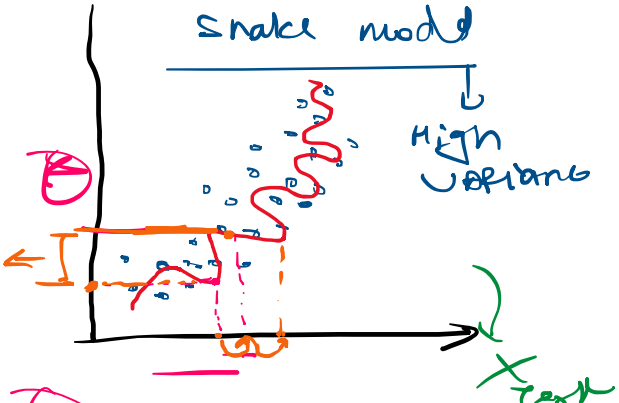
× unable to generalize



Training DATA

Low VARIANCE

(too simple)



Snake model

High VARIANCE

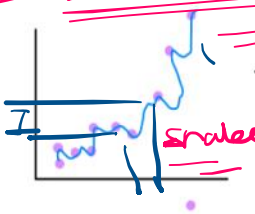
* High VARIANCE

* (too complex)

VARIATION plot

ideally → Low Bias + low VARIANCE

* 3 scenarios



degree ↑

→ overfit : Low Bias → TRAIN ↑

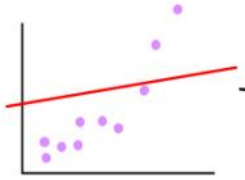
+ High Variance → TEST ↓

Snake → inconsistency

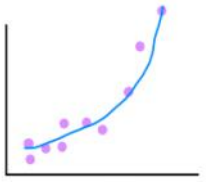
* Bias → training

VARIANCE → TEST

VARIANCE → Test



→ Underfit : High Bias → Train ↓ + Low Variance → Test ↑



→ Perfect fit : Low Bias → Train ↓ + Low Variance → Test ↓

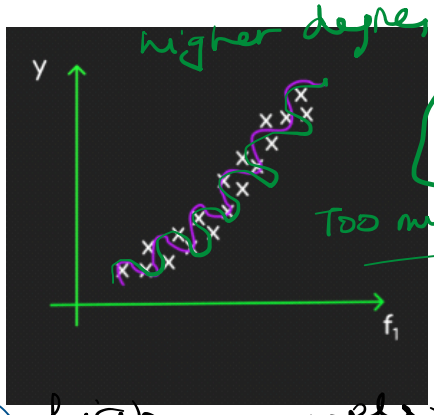
High variance

Polynomial Feature

↳ $\hat{y} = w_0 + w_1 f_1 + w_2 f_1^2$

(*) Regularization → Sol^n to OVERFIT + UNDERFIT

degree = 2



$\hat{y} = w_0 + w_1 f_1 + w_2 f_1^2 + w_3 f_1^3 + w_4 f_1^4 + w_5 f_1^5$

(*) high complexity ↓

Polynomial features

degree = 5

↓

overfitting

Low Bias + High VARIANCE

→ OVERFITTING

degree ⟹ optimal ok

* Regularization ? → Reduce influence of higher degree or some of the features

if $w_3 = w_4 = w_5 = 0.00001$ very small

↳ conclusion ?

* $\hat{y} = w_0 + w_1 f_1 + w_2 f_1^2$ ⇒

* good eqn

* Balanced

⇒ loss of → FF, degree

* Shrinkage features → new + polynomial

* Automated → Regularization

$w_3 = w_4 = w_5 = 0$ & $w_1, w_2 \neq 0$

* some good no

How Reg^n decides that only $w_3\ w_4\ w_5$ should be 0. [why not $w_1, w_2 = 0$] ?

↓

MC → Loss

* Modified loss function :- ⇒

Sq. of all the weights

$$\text{Loss} = \left(\frac{1}{n} \sum_{i=1}^{n} \left(y^{(i)} - \hat{y}^{(i)} \right)^2 \right) + \sum_{j=1}^{d} w_j^2$$

MSE + Regularization

→ penalizing the weights

↓

Some wts = 0

Some ≠ 0

Loss = MSE + Regularization

updated loss

$$\Rightarrow \underset{w_1, w_0}{\arg\min} \left[\text{MSE} + \text{Regularization} \right]$$

f1 loss

GD

Case I

$$\hat{y} = w_1 f_1 + w_2 f_1^2 + w_3 f_1^3 + w_4 f_1^4 + w_5 f_1^5$$

$w_1 = 2$
$w_2 = 1.5$
$w_3 = 1.8$
$w_4 = -0.4$
$w_5 = 1.4$

→ complicated

all features contributing

w_{100}

MSE → happy, Reg^n → penalty added

very complex

wrong

→ overfitting

Case II

$w_1 = w_2 = w_3 = w_4 = w_5 = 0$

$\hat{y} = w_0$

underfit model

→ MSE → sad, Reg^n → happy

Loss → MSE sad, happy

→ Reg^n

MSE purpose ⇒ Create complex model such that error can be reduced.

→ To Reduce Bias

Overfit ← underfit →

Regularization purpose ⇒ adds penalty for large weights

→ TRIES to keep weights small

→ Reduces overfitting by penalizing large wts & encouraging simple models

- Both terms have diff. OBJECTIVES

MSE → wants to Reduce Bias

Reg → wants to Reduce variance

↳ Both Happy → common ground

More penalty

* How to make both terms happy ? ⇒ Come to some middle ground.

* $\text{Loss} = \frac{1}{n}\sum_{i=1}^{n}(y^i - \hat{y}^i)^2 + \lambda \sum_{j=1}^{d} w_j^2$

  - λ → Regularization strength
  - $\frac{1}{n}\sum_{i=1}^{n}(y^i - \hat{y}^i)^2$ → MSE
  - $\lambda \sum_{j=1}^{d} w_j^2$ → acts as a tuner → Reg°

$\lambda > 0$ → decides complexity

* if $\lambda = 0$

if $\lambda = 10^8$ → very large value

→ Reg^r → more imp

→ MSE wins here

→ lower the error
Bias

→ complex model

→ overfitting

→ all features participating

→ Regr → more imp

$(w_1\ w_2\ w_3\ w_4\ w_5) \geq 0$

Loss = MSE + $10^5 \times (w_0^2)$

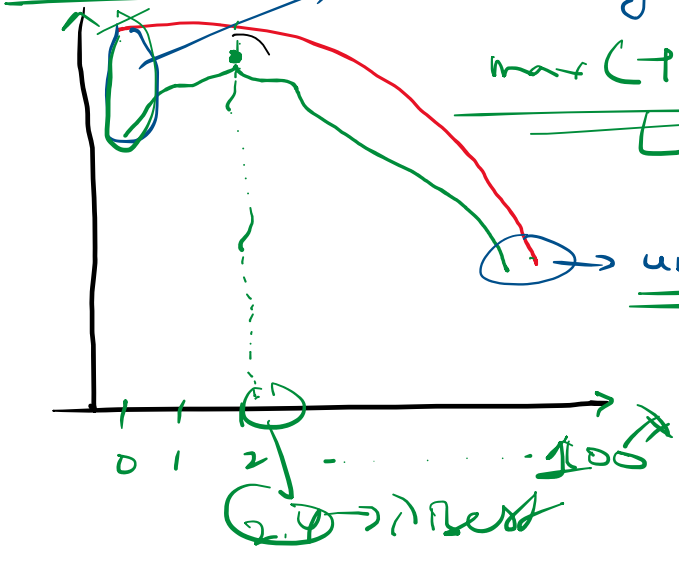
Huge Loss

→ min Loss

if $\alpha \uparrow \geq$ ⇒ model underfits

$\downarrow \alpha$ ⇒ model overfits

Performance



max (test) → Best

Best

opt. ⇒

force wts to become zero

all wts → 0

$\hat{y} = w_0$

too simple model → underfit

Default

$x \rightarrow (1, 1000)$

⇒ Hyperparam tuning ⇒ MSE Takes Power

$\alpha = 0$ ⇒

Range → (0, 1)

↑ λ ⇒ $\lambda = 10$

Contract B/w MSE & Regularization

↑ features ⇒ ✗ [λ = Reasonable value]

$w_3 = w_4 = w_5 = 0$ $w_1, w_2 \neq 0$

$\lambda = 0$ → MSE low → overfit ✗

↑ λ → $\lambda = 10$ MSE & Reg" → Contract

Types of Regularization ⇒

✗ Loss = MSE + $\lambda \sum_{j=1}^{d} w_j^2$ L2 Regularization

$\lambda \sum_{i}^{d} |w_i|$ ⇒ (L1)

$\lambda \sum_{j=1}^{d} |w_j| \Rightarrow$ L1 Regularization

Interesting property of L1 and L2 :

⇒ L1 :

1. Since L1 makes wts of useless features = 0
   Hence used in feature selection
2. As L1 creates sparse weight vector, it reduces the number of non-zero parameter leading to Efficient memory usage and faster computation in high dimensional datasets

features = [0 0 0]

⇒ L2 : $\Rightarrow w_j^2$

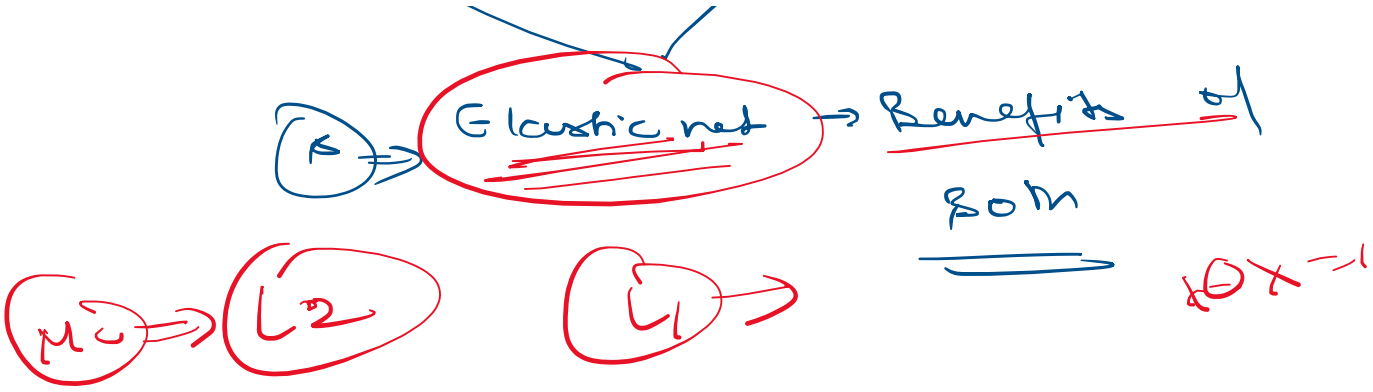
1. The square penalty term w_j^2 encourages smoother parameter values → beneficial in case of noisy data
2. L2 tends to net zero out weight value of less useful features ⇒ helpful when we want model to aim for a balanced use of features

ElasticNet ⇒ $L_1 + L_2$ → [0.00 ... 1] ✗

$$\text{Loss} = \text{MSE} + \lambda_1 \sum_{j=1}^{d} |w_j| + \lambda_2 \sum_{j=1}^{d} w_j^2$$

L1 → too aggressive
→ Removes features completely
✗ ✗ 0

L2 → too soft
→ Shrinks weights but keeps almost all features



L1 → pushes some wts exactly to 0

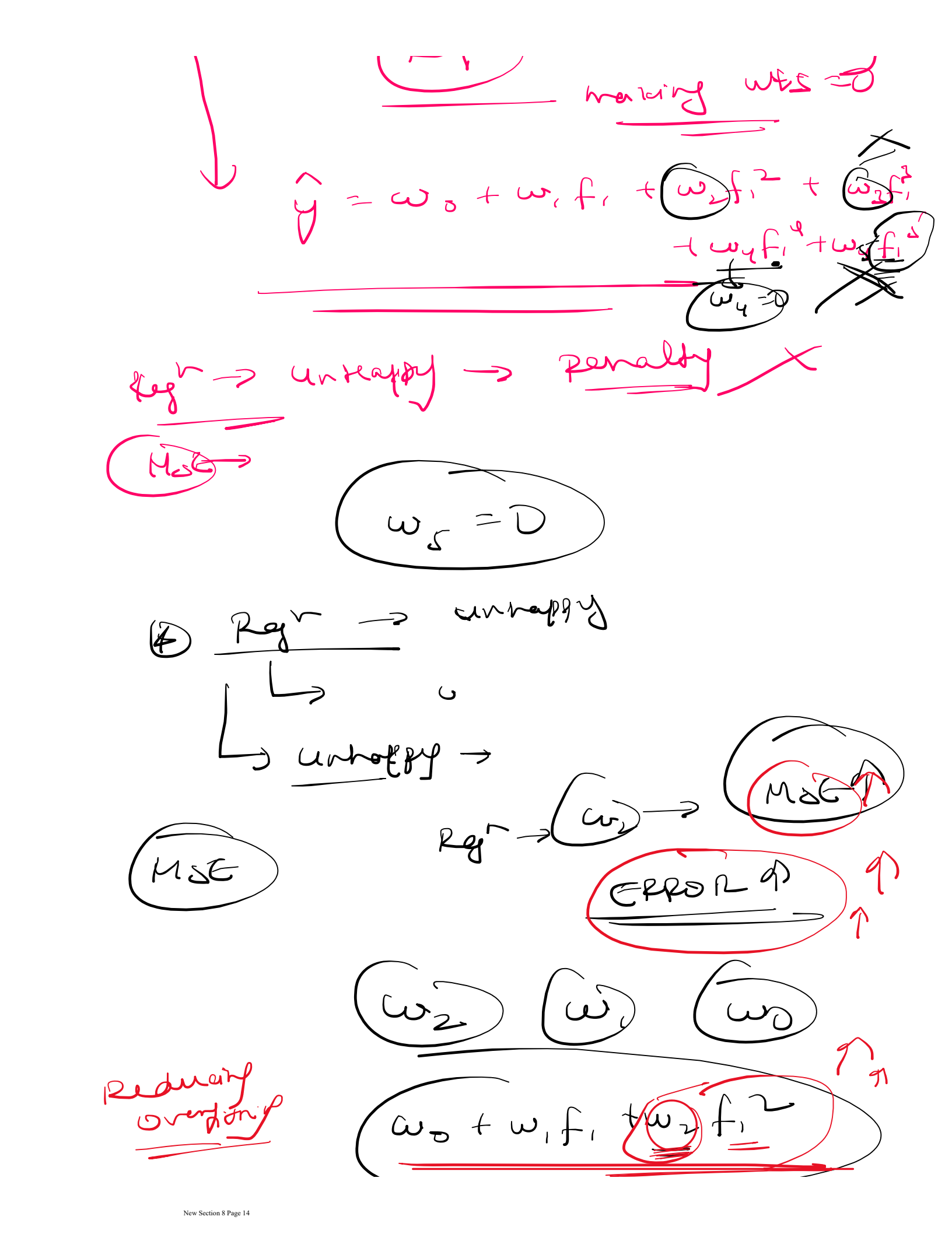
L2 → keeps wts small
→ Reduces variance
→ Handles correlated features better

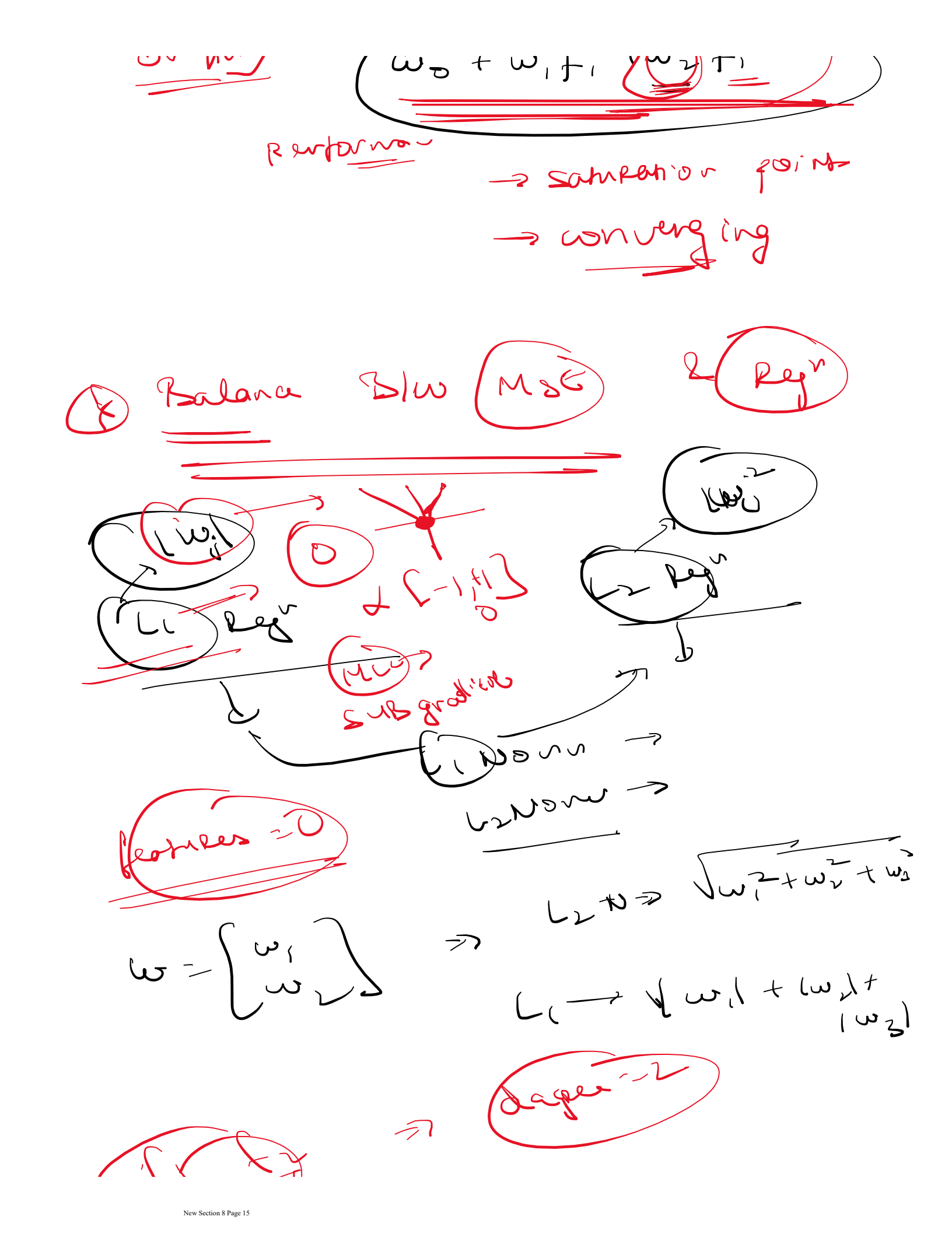
④ Elastic Net ⇒ keep model simple,
Shrink weights smoothly
But also Remove unnecessary features

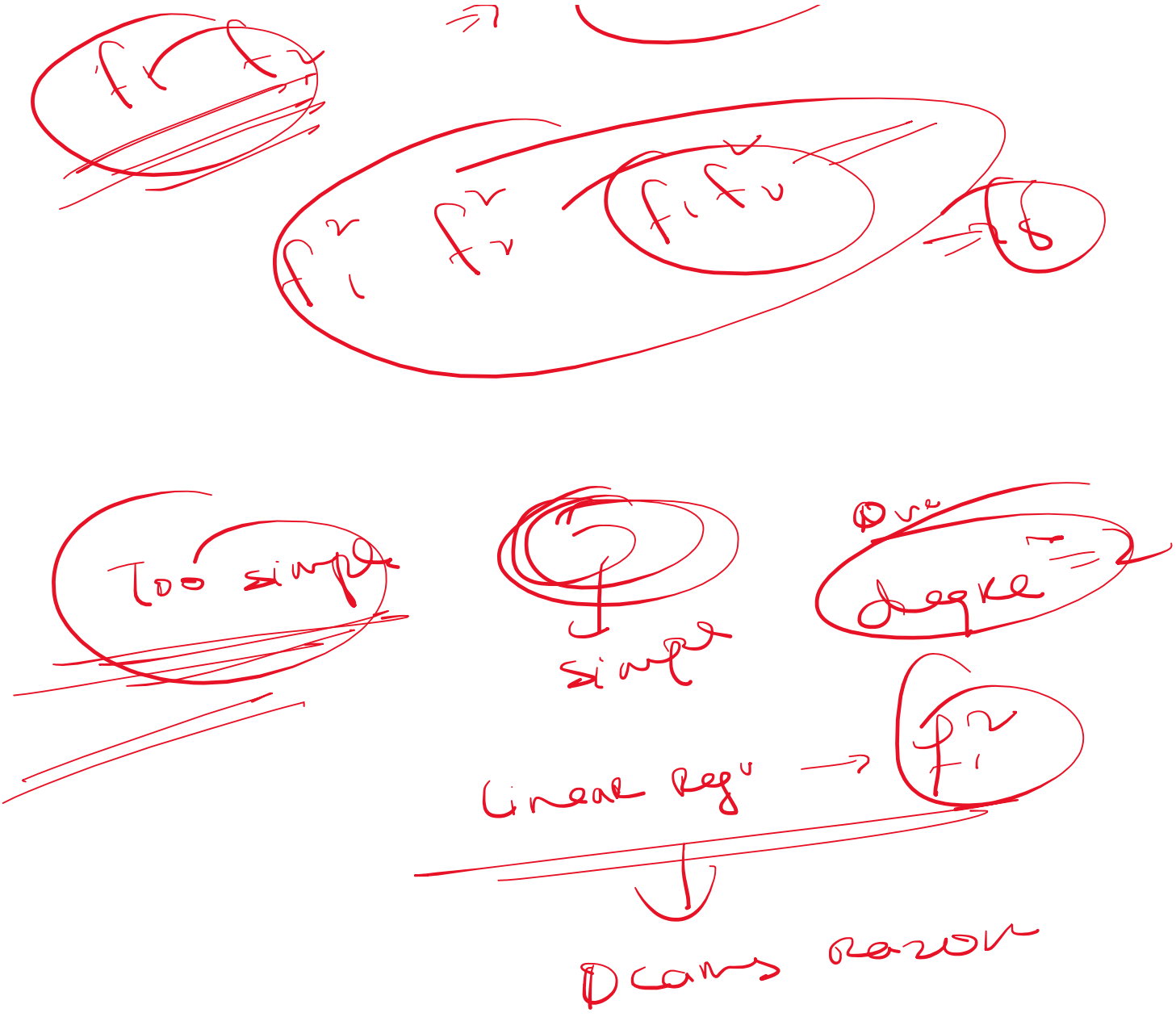
Contract

MSE → use all features
Reduce Bias

Reg^n → min. variance by making wts ≈ 0


making wts = 0
$\hat{y} = w_0 + w_1 f_1 + w_2 f_1^2 + w_3 f_1^3 + w_4 f_1^4 + w_5 f_1^5$
$w_4 = 0$
Reg^r → underfit → Penalty ✗
MSE →
$w_5 = 0$
Reg^r → underfit
Reg^r → w_2 →
MSE
MSE
ERROR ↑
w_2 w_1 w_0
$w_0 + w_1 f_1 + w_2 f_1^2$
Reducing overfitting


Regression
→ saddle point
→ converging
Balance b/w MSE & Reg^n
L1 Reg^n
L2 Reg^n
sub gradient
L1 norm →
L2 norm →
features = 0
L2 N ⇒
L1 →
degree = 2



Parameters → w_0, w_1 ⇒ learnt By the model during training

Hyperparams ⇒ d , λ_1 , λ_2, η

λ → Train for multiple values of λ

0.1 → 0.5 → 10

train(), test()

for λ in range(0.1, 10, 0.5)

model = lasso(λ)

model.fit(Train)

Train perf → store

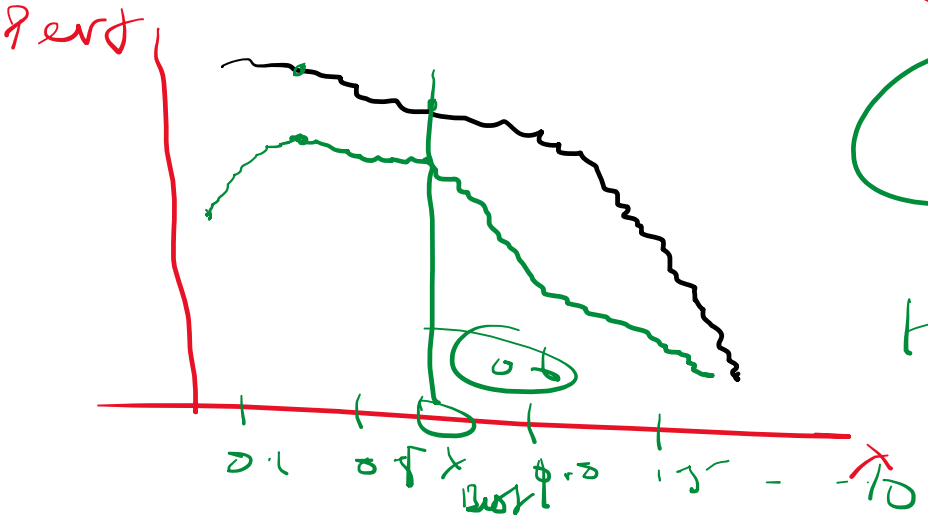
test perf

[ ]

[ ] → test

→ Best

Test → curves



Hyperparam Tuning

Scratch → framework

Hyperparam tuning

Cross validation



→ hidden Layers

assess the performance

Manager

Best λ

M

Hyperparam tuning

Train → learn the pattern

Test

$\lambda = 0.18$

78%

0.79

0.2

0.82

0.6

0.86

0.89

$\lambda_{best} = 0.6$

multiple times

Test → 86%

Mr → Prod

Expectation → 86%

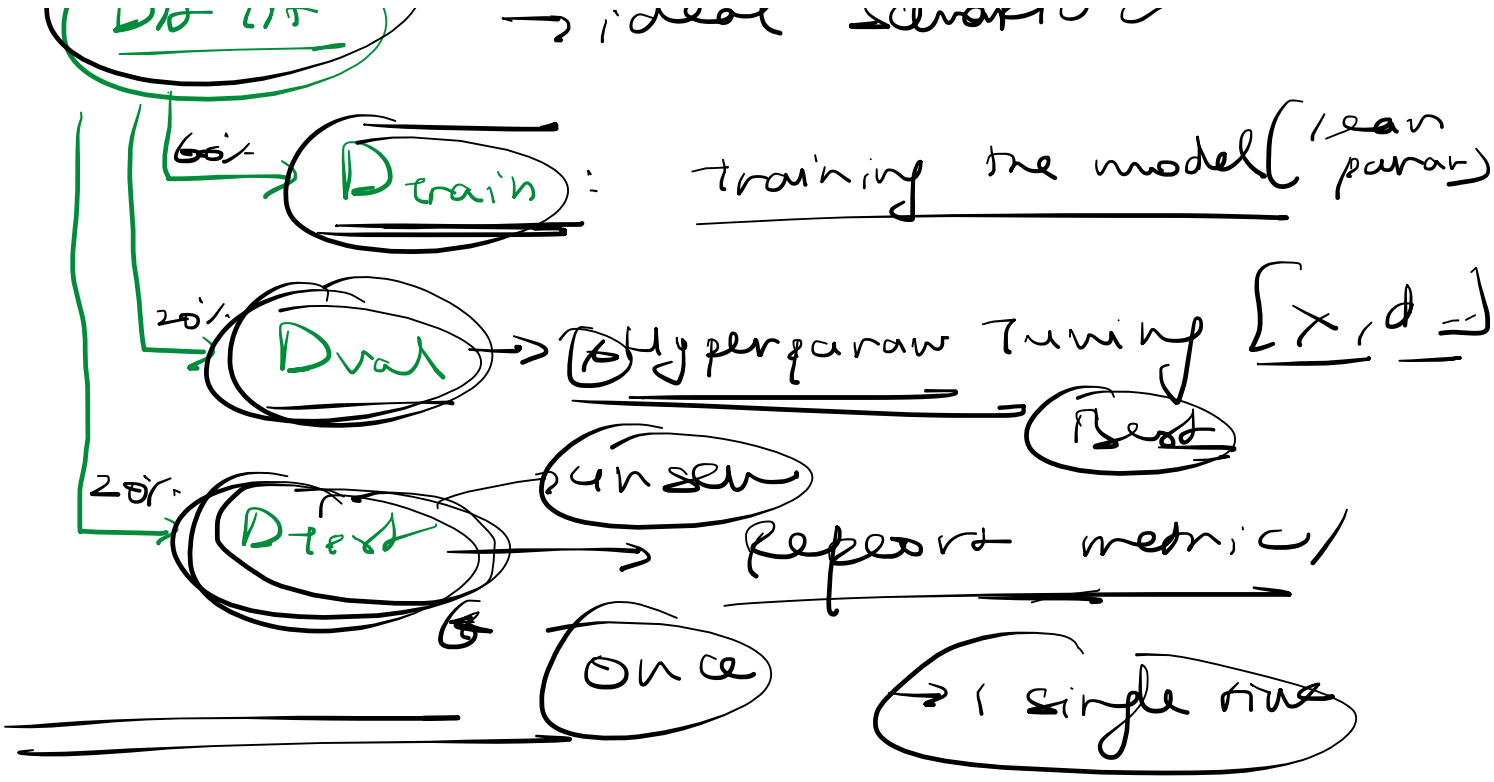
Customer facing → 82%

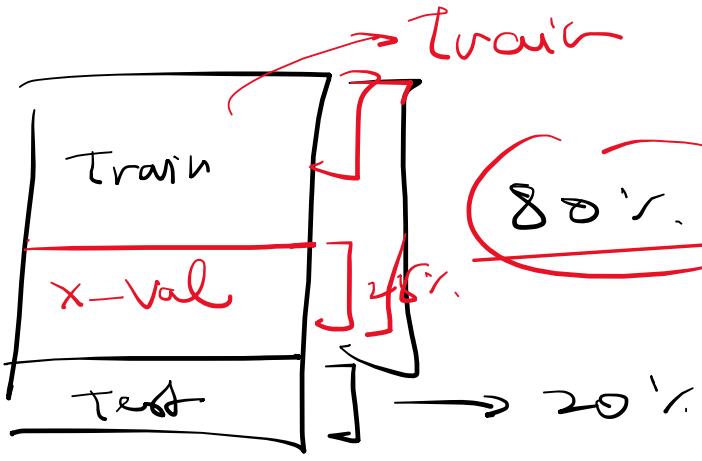
X → problem

Test → unsem

Single Time

DATA → ideal scenario





0.8 x 0.25

0.2

K fold

next class

next